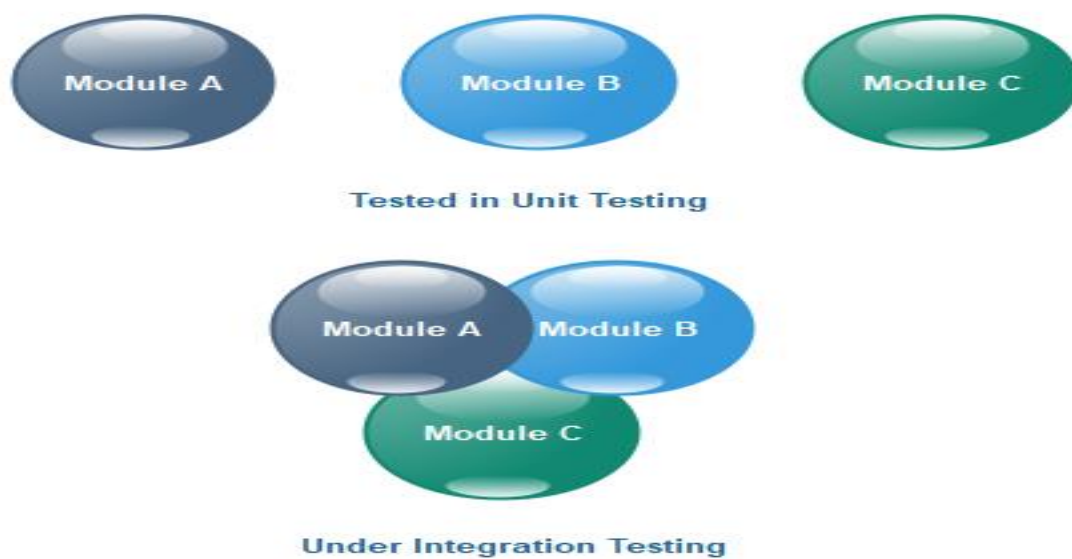


Integration testing

1. Integration testing is the second level of the software testing process comes after unit testing.
2. In this testing, units or individual components of the software are tested in a group.
3. The focus of the integration testing level is to expose defects at the time of interaction between integrated components or units.
4. Unit testing uses modules for testing purpose, and these modules are combined and tested in integration testing. The Software is developed with a number of software modules that are coded by different coders or programmers. The goal of integration testing is to check the correctness of communication among all the modules.



- Once all the components or modules are working independently, then we need to check the data flow between the dependent modules is known as **integration testing**.

Guidelines for Integration Testing

- We go for the integration testing only after the functional testing is completed on each module of the application.
- We always do integration testing by picking module by module so that a proper sequence is followed, and also we don't miss out on any integration scenarios.
- First, determine the test case strategy through which executable test cases can be prepared according to test data.
- Examine the structure and architecture of the application and identify the crucial modules to test them first and also identify all possible scenarios.
- Design test cases to verify each interface in detail.
- Choose input data for test case execution. Input data plays a significant role in testing.
- If we find any bugs then communicate the bug reports to developers and fix defects and retest.

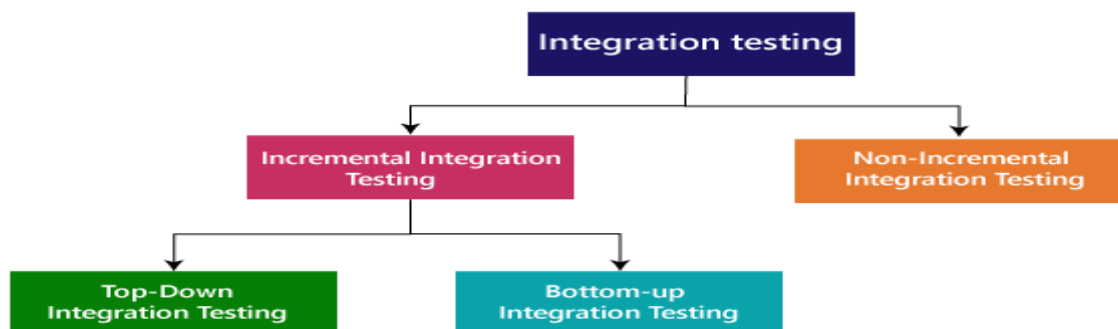
- Perform **positive and negative integration testing**.

Reason Behind Integration Testing

- Although all modules of software application already tested in unit testing, errors still exist due to the following reasons:
 1. Each module is designed by individual software developer whose programming logic may differ from developers of other modules so; integration testing becomes essential to determine the working of software modules.
 2. To check the interaction of software modules with the database whether it is an erroneous or not.
 3. Requirements can be changed or enhanced at the time of module development. These new requirements may not be tested at the level of unit testing hence integration testing becomes mandatory.
 4. Incompatibility between modules of software could create errors.
 5. To test hardware's compatibility with software.
 6. If exception handling is inadequate between modules, it can create bugs.
- Any testing technique (Blackbox, Whitebox, and Greybox) can be used for Integration Testing

Types of Integration Testing

- Integration testing can be classified into two parts:
- **Incremental integration testing**
- **Non-incremental integration testing**



In the Incremental Approach, modules are added in ascending order one by one or according to need. The selected modules must be logically related. Generally, two or more than two modules are added and tested to determine the correctness of functions. The process continues until the successful testing of all the modules.

OR

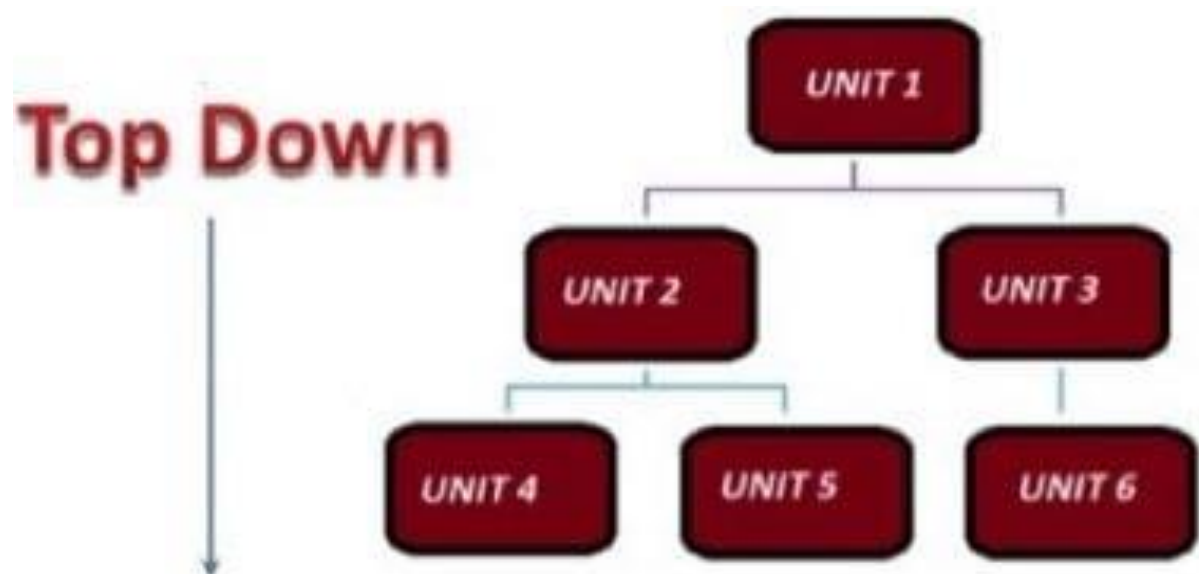
- In this type of testing, there is a strong relationship between the dependent modules. Suppose we take two or more modules and verify that the data flow between them is working fine. If it is, then add more modules and test again.

For example: Suppose we have a Flipkart application, we will perform incremental integration testing, and the flow of the application would like this:

- Flipkart → Login → Home → Search → Add cart → Payment → Logout
- Incremental integration testing is carried out by further methods:
- Top-Down approach
- Bottom-Up approach

Top-Down Approach

- The top-down testing strategy deals with the process in which higher level modules are tested with lower level modules until the successful completion of testing of all the modules. Major design flaws can be detected and fixed early because critical modules tested first. In this type of method, we will add the modules incrementally or one by one and check the data flow in the same order.

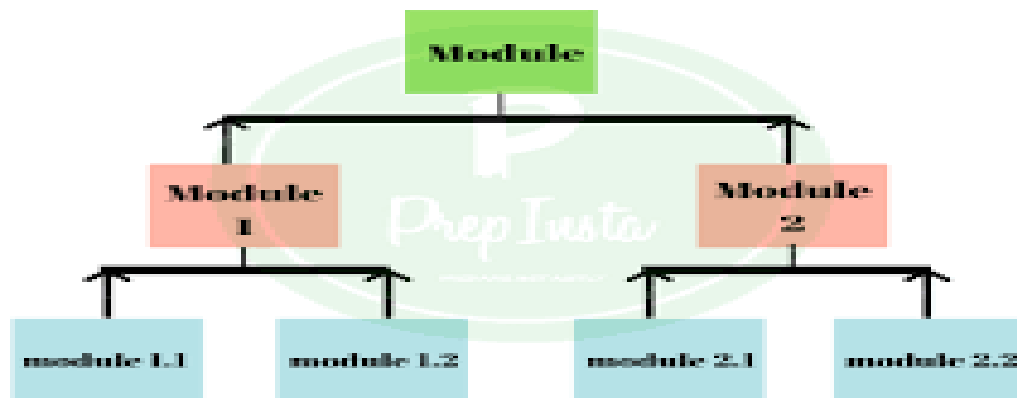


Bottom-Up Method

The bottom to up testing strategy deals with the process in which lower level modules are tested with higher level modules until the successful completion of testing of all the modules. Top level critical modules are tested at last, so it may cause a defect. Or we can say that we will be adding the modules from **bottom to the top** and check the data flow in the same order.

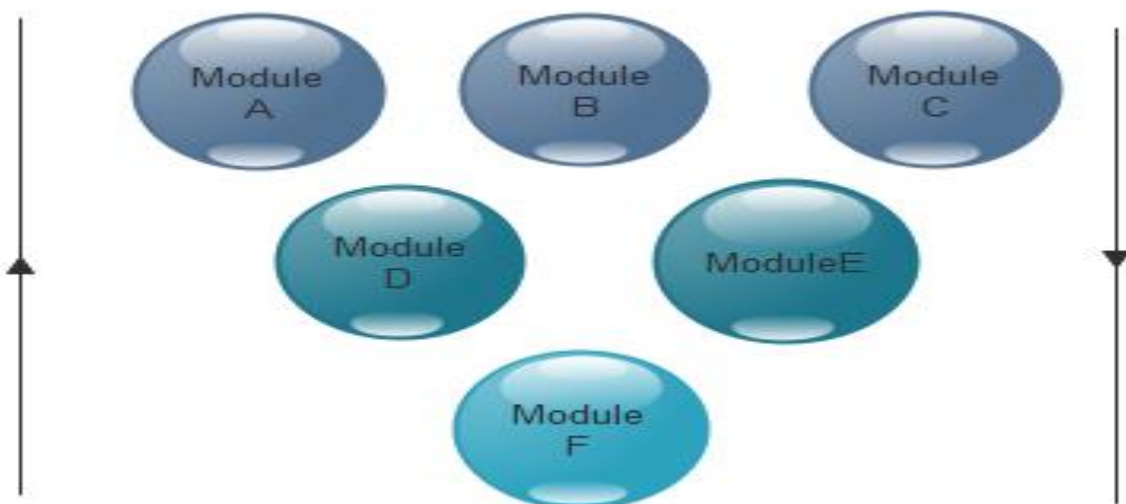


Bottom Up Approach



Hybrid Testing Method

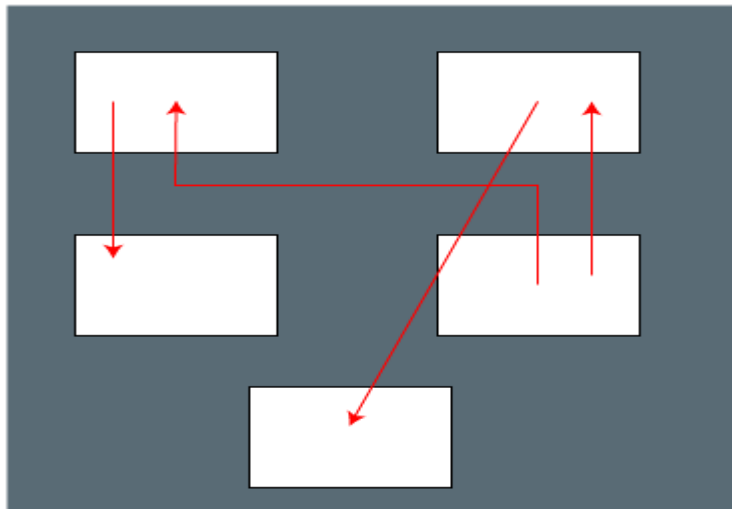
- In this approach, both **Top-Down** and **Bottom-Up** approaches are combined for testing. In this process, top-level modules are tested with lower level modules and lower level modules tested with high-level modules simultaneously. There is less possibility of occurrence of defect because each module interface is tested.



Hybrid Method

Non- incremental integration testing

- We will go for this method, when the data flow is very complex and when it is difficult to find who is a parent and who is a child. And in such case, we will create the data in any module bang on all other existing modules and check if the data is present. Hence, it is also known as the **Big bang method**.



STUBS AND DRIVERS

- Stubs and drivers are two types of test harness that are a collection of software and test, which means they are designed together to test a unit of a program by accelerating the change of situations at the same time as regularly checking its performances and results.
- ***In software testing, a test harness or automated test framework is a collection of software and test data
- In the Software Testing process, the Stubs and Drivers are those components that are used as a short-term alternative for a module.
- In Software Testing the words **stub and drivers** described as a replica of the modules that operate as an alternative to the new or missing modules.
- Stubs are mainly used in **top-down integration testing** on the other hand, drivers are mainly used in **bottom-up integration testing** individually and designed the enhance the testing process.
- To sustain the essential requirements of the inaccessible modules or components, we are precisely established the stubs and drivers.
- **A stub** is a replica of a module that collects the data and develops many possible data. However, it executes like an actual module and is mainly used to test modules.
- Generally, stubs are created by software developers in order to use them instead of modules if the particular modules are miss or not yet developed.
- By using these test stubs, the test engineers can simulate the performance of the lower-level modules, which are not yet joined with the software. Furthermore, it helps us to accelerate the activity of the missing modules.

Types of Stubs

- In the **top-down approach of incremental** integration testing, the stubs are divided into four essential parts, which are as follows:
- Demonstrate the trace message.
- Exhibits the parameter values.
- Returns the consistent values, which are handled by the modules or the components.
- Returns the values of the specific parameters, which were utilized by testing components or modules.
- What are Drivers?
- The **Drivers** establish the test environments and takes care of the communication, estimates results, and also sends the reports.
- These are just like stubs and used by software test engineers in order to accomplish the missing or incomplete modules/ components requirements.
- The drivers are mainly developed in the **Bottom-up approach of incremental integration testing**.
- Generally, drivers are bit complex as compared to the stubs.
- These can test the lower levels of the code when the upper-level modules or codes are not developed or missing.
- In other words, we can say that the Drivers perform as **pseudo-codes**, which are mainly used when the stub modules are completed; however, the initial modules/components are not prepared.

Example

- we have one web application that contains **four different modules**, such as:
- **Module-P**
- **Module-Q**
- **Module-R**
- **Module-S**

Modules	Individual Activities
Module p	Login Page of WebApp
Module Q	Home page of WebApp
Module R	Print setup
Module S	logout

- It is always a better approach to implement the testing or development of all the modules equivalently. The minute each gets developed, they can be combined and tested according to their similar dependencies with a module.

- Once **Module-P** is developed, it will go through the testing process. But, to perform and validate the testing methods regarding **Module-P**, they need **Module-Q**, which is not yet developed entirely and still in the developing process.
- And it is not possible to test **Module-P** on the lack of **Module-Q**. Thus, in such scenarios, we will take the help of **Stubs and Drivers** in the **software testing process**.
- The Stubs and drivers will replicate all the basic functionality and features displayed by the real **Module-Q**. And subsequently, it is being combined with **Module-P** in order to execute the testing process effectively.
- Now, we can validate the estimated functionality of the **Login page**, which was in **Module-P**, only if it is going to the **Home Page**, which is the activity of **Module-Q** as per the correct and valid inputs.
- In the same way, **stubs and drivers** are used to accomplish the requirements of other modules, such as the **Sign-out page**, which comes under the Module-S and needs to be aimed to the **Login page (Module-P)** after effectively logging out from the particular application.
- Likewise, we may also use Stubs or Drivers instead of **Module-R** and **Module-S** if they are not available
- As the result of the inaccessibility of **Module-P**, stubs and drivers will work as an alternate for it to perform the testing of **Module-S**.

The key difference between Stubs and Drivers

- **Stubs and Drivers** design as a dummy for the missing or inaccessible modules or the components.
- Most commonly, stubs and drivers are used in the **incremental integration testing** where stubs are used in **top-bottom methodology** while drivers are used in a **bottom-up methodology**.
- Usually, developers and unit test engineers are included in the creation of stubs and drivers.
- Even though it provides an ease to perform the **separate components or the modules** without worrying about the accessibility of other modules, and leads them into a time-consuming process, as it involves the development of replica for all the missing modules.
- Precisely, stubs and drivers are developed for each module holding different purposes.

	STUB	Driver
Type	Dummy codes	Dummy codes
Description	Routines that don't actually do anything except declare themselves and the parameters they accept. The rest of the code can then take these parameters and use them as inputs	Routines that don't actually do anything except declare themselves and the parameters they accept. The rest of the code can then take these parameters and use them as inputs
Used in	Top Down Integration	Bottom Up Integration
Purpose	To allow testing of the upper levels of the code, when the lower levels of the code are not yet developed.	To allow testing of the lower levels of the code, when the upper levels of the code are not yet developed.